Models used in the framework, and why they were chosen

Domain-agnostic federated neuro-symbolic framework

Reliable, explainable LLM reasoning over privacy-preserving heterogeneous data

1. Overview: a five-layer pipeline, not a single model

The framework deliberately combines several models, each responsible for one job. The neural model gives accuracy, the federated optimiser lets it learn without moving raw data, the symbolic engine gives verifiable logic, and the explainer turns verified facts into safe text. This separation is what makes the system both accurate and reliable.

Layer	Component / model	Type	Role
1	PerceptionMLP	Neural network (PyTorch)	Classify tabular records per domain
4	FedAvg / FedProx / FedAdagrad	Federated optimisers	Train Layer 1 across clients, no raw data shared
2	NeuralSymbolicBridge	Deterministic converter	Turn predictions + features into typed facts
3	SymbolicReasoner	Rule engine	Verify predictions, apply safe overrides
5	VerifiedLLMExplainer	Constrained generator + guard	Explain using verified facts only

2. Neural model: PerceptionMLP (Layer 1)

A compact multi-layer perceptron defined in nsf/neural/models.py. The same architecture is reused for both domains (only the input and output sizes change), which is what makes the framework domain-agnostic.

Architecture

Stage	Operation	Purpose
Input	Linear(input_dim -> 96)	Project features into a hidden space
	LayerNorm(96) -> ReLU -> Dropout(0.12)	Stabilise + regularise
Hidden	Linear(96 -> 96)	Learn feature interactions
	LayerNorm(96) -> ReLU -> Dropout(0.08)	Stabilise + regularise
Output	Linear(96 -> num_classes)	Class logits (softmax for probabilities)

Healthcare uses input_dim=12 and num_classes=3 (low/medium/high risk). Banking uses input_dim=14 and num_classes=2 (legitimate/fraudulent).

Why an MLP, and why these choices

- The data is small, tabular and heterogeneous. For tabular data an MLP is a strong, standard baseline; CNNs assume spatial structure and Transformers need far more data, so neither is justified here.
- LayerNorm (not BatchNorm) is used because federated clients have small, non-IID batches. BatchNorm statistics are unstable and leak batch information across the federation; LayerNorm normalises per-sample and is federation-safe.
- Dropout (0.12 then 0.08) regularises the model so it does not overfit the small per-client datasets.
- It is compact (one hidden width of 96) so each training round is fast on CPU and the parameter payload sent each federated round stays small (about 5 MB total communication per strategy in our runs).
- A simple, inspectable model keeps the downstream symbolic verification meaningful: we can reason about

its outputs rather than treat it as an opaque black box.

3. Federated training optimisers (Layer 4)

Training is federated: each client trains locally on its own data and only model parameters are aggregated on the server. Raw records never leave the client (nsf/federated/client.py). Clients optimise with SGD (momentum 0.9) and cross-entropy loss. Three server strategies are implemented and compared so we can show which behaves best under non-IID data (nsf/federated/aggregation.py).

Strategy	What it does	Why it is included
FedAvg	Size-weighted average of client weights.	The canonical baseline (McMahan 2017); communication-efficient reference point.
FedProx	FedAvg plus a client-side proximal term (mu/2)||w - w_global||^2.	Handles non-IID / system heterogeneity by limiting client drift (Li 2020).
FedAdagrad	Adaptive server optimiser: accumulates squared pseudo-gradients and scales the update.	Faster, more robust convergence under heterogeneity (Reddi 2021).

- Why three and not one: the datasets are split across clients with a non-IID Dirichlet partition (alpha 0.6-0.7), so client distributions differ a lot. Comparing FedAvg, FedProx and FedAdagrad demonstrates robustness and lets the project report which converges best per domain.
- Why SGD with momentum on the client: it is the standard, stable local optimiser used in the FedAvg/FedProx/FedAdagrad literature and keeps client compute light.
- FedProx's proximal term is applied on the client; its server aggregation is identical to FedAvg. FedAdagrad keeps an extra server-side accumulator, which is why its server update differs.

4. The non-neural models and why they matter

Neural -> Symbolic Bridge (Layer 2)

A deterministic converter (nsf/bridge/neural_symbolic.py) that packages the prediction, confidence and the numeric/boolean features into a typed SymbolicFact. Why: it guarantees that only structured facts - never free-form text - enter the logic layer, which is the first line of defence against hallucination.

Symbolic Reasoner (Layer 3)

A transparent rule engine (nsf/symbolic/engine.py) that scores published domain rules, compares the symbolic class with the neural class, and applies an override only under hard safety constraints or when neural confidence is low. Why a rule engine instead of a second network: its decisions are auditable, reproducible, and provide the evidence sentences the explainer is allowed to use. (The thresholds behind these rules are justified in NSF_Threshold_Evidence.pdf.)

Verified LLM Explainer (Layer 5)

A constrained, fact-locked generator with a hallucination guard (nsf/llm/explainer.py). It consumes only verified facts and can mention only rules that actually fired; if any drift is detected it falls back to a strict template. Why not a raw external LLM: reliability and reproducibility without an API key. An external API model could later be plugged in purely as a rephraser over the same verified payload, without gaining the ability to invent facts.

5. Training configuration (configs/default.yaml)

Setting	Value	Setting	Value
Clients	5	Global rounds	12
Local epochs	4	Batch size	32
Learning rate	0.015	FedProx mu	0.01
Client optimiser	SGD (momentum 0.9)	Loss	Cross-entropy
Device	cpu (cuda optional)	Seed	42

6. Measured results (from results/summary.json)

Domain	Strategy	Neural acc	Neuro-sym acc	Comm (MB)	Halluc-free
healthcare	fedavg	90.7%	88.0%	5.14	100%
healthcare	fedprox	91.3%	89.3%	5.14	100%
healthcare	fedadagrad	92.7%	90.0%	5.14	100%
banking	fedavg	99.0%	99.0%	5.19	100%
banking	fedprox	98.8%	98.8%	5.19	100%
banking	fedadagrad	98.8%	99.0%	5.19	100%

Best strategy - healthcare: fedadagrad (neuro-symbolic accuracy 90.0%). Best strategy - banking: fedavg (neuro-symbolic accuracy 99.0%). The hallucination-free ratio is 100% in every configuration, which is the main reliability claim of the design.

7. Why this combination of models

- Accuracy: the federated MLP reaches ~91-93% (healthcare) and ~99% (banking) on held-out test data.
- Privacy: federated learning keeps raw client data local; only parameters are shared.
- Reliability: the symbolic engine verifies every prediction and can override unsafe ones.
- Explainability: the explainer is fact-locked, so explanations never introduce unverified claims.
- Domain-agnostic: the identical neural architecture and pipeline serve both Healthcare and Banking by changing only the input/output sizes and the domain rule set.

8. Method references

- McMahan B, et al. Communication-Efficient Learning of Deep Networks from Decentralized Data (FedAvg). AISTATS 2017. https://arxiv.org/abs/1602.05629
- Li T, et al. Federated Optimization in Heterogeneous Networks (FedProx). MLSys 2020. https://arxiv.org/abs/1812.06127
- Reddi S, et al. Adaptive Federated Optimization (FedAdagrad/FedAdam/FedYogi). ICLR 2021. https://arxiv.org/abs/2003.00295
- Ba JL, Kiros JR, Hinton GE. Layer Normalization. 2016. https://arxiv.org/abs/1607.06450
- Srivastava N, et al. Dropout: A Simple Way to Prevent Neural Networks from Overfitting. JMLR 2014.